

Improving a Matrix Multiplication Bound with a MacBook and a Proof Assistant

BSD (bsd.developer@proton.me)
Independent Researcher

25 August 2026

Abstract

We give the first machine-checked bound on the matrix multiplication exponent: a Lean 4 theorem, quantified over 1.5 MB of published canonical certificate bytes, establishing $\omega \leq 10935605172023554189/2^{62} = 2.371281376 \dots$, below the best published level-three bound by 5.76×10^{-5} . The trusted base is two enumerated axioms: the published feasibility-to- ω theorem, and one native evaluation (profile CN) corroborated by an independent Rust checker. The numerical search is untrusted and unpublished by design; the certificate is the entire argument. A stronger level-four bound (2.371177) is reported without formal verification; we do not claim the state of the art.

1. The gap between computer-assisted and machine-checked

Since Strassen showed that matrix multiplication is not cubic (Strassen 1969), the upper bound on ω has been improved by a sequence of results resting on the laser method (Coppersmith and Winograd 1990; Gall 2014), most recently through its refinement into *combination-loss analysis* (Duan, Wu, and Zhou 2023; Williams et al. 2024; Alman et al. 2025; Dupont et al. 2026). The modern form of these results is uniform: a theorem states that any feasible solution of a stated optimization problem yields an upper bound on ω , and a computer search exhibits a solution.

The solution is a large numerical object — in the instance considered here, a tree with 5,779 nodes and 762 maximum-entropy witnesses — and the assertion that it is feasible is a claim about a computation over that object. Every published bound in this line is therefore *computer-assisted*: correctness depends on a program, and the reader is asked to trust that program.

This is not a defect in those results, but it does leave a gap, and closing it is what this paper is about. We distinguish:

- **computer-assisted**: a program was run, and its output is reported;
- **machine-checked**: a proof assistant’s kernel has accepted a term whose type is the claim, and the assumptions that term depends on can be enumerated mechanically.

The second makes the logical reduction and its assumptions mechanically enumerable. Related proof-assistant treatments exist for other computer-assisted mathematics, including the four-colour theorem (Gonthier 2008) and the Kepler conjecture (Hales et al. 2017).

To our knowledge, no bound on ω has been checked in a proof assistant. We state this carefully because it is a negative literature claim: none of (Duan, Wu, and Zhou 2023; Williams et al. 2024; Alman et al. 2025; Dupont et al. 2026) reports a proof-assistant development, and (Dupont et al. 2026) describes verification code rather than a formal theorem. We are not aware of an independent formalization.

1.1 Contributions and scope

The paper makes four contributions.

A proof-carrying result architecture. A reportable result is not a floating-point score but a bundle: canonical exact certificate bytes, the generated theorem module, and the axiom report. Verification requires neither the search that found the point nor the program that wrote the certificate; both stay outside the trusted base, and the released repository contains everything the reader needs to check the claim.

A byte-parametric Lean soundness chain. A total decoder and a directed exact checker prove that acceptance of canonical certificate bytes implies the defined combination-loss feasibility predicate. Certificate-specific results instantiate that theorem at literal bytes, eliminating an unaudited transcription between the downloaded artifact and the formal claim.

A CN-certified level-three improvement. Lean’s kernel accepts a theorem establishing $\omega \leq 10935605172023554189/2^{62} = 2.371281376 \dots$. Under profile CN, the concrete acceptance equality is supplied by one declared native-evaluation axiom. Relative to the displayed threshold, the result improves on the $\omega < 2.371339$ of (Alman et al. 2025) by 5.76×10^{-5} at the same recursion level and in the same published feasibility formulation.

An implementation-diverse exact cross-check. A separately implemented Rust checker evaluates the same specified acceptance claim over the same certificate bytes. Its agreement is corroborating evidence, not a second proof and not part of the Lean theorem’s assumptions.

We do **not** claim the state of the art. The preprint (Dupont et al. 2026) reports $\omega < 2.371177$, obtained at level four, and our bound is 1.04×10^{-4} above it. §7 details the gap.

We also do not claim new mathematics. Combination-loss analysis is (Duan, Wu, and Zhou 2023), and the optimization problem is stated in (Alman et al. 2025). What we contributed is a verified reduction and a stronger level-three feasible point. §7.2 is explicit about the search provenance, because the improvement over (Alman et al. 2025) is an optimization result rather than a new mathematical analysis.

The theorem certifies the selected endpoint, not the historical execution of any optimizer, and this release deliberately excludes the search apparatus: its correctness, convergence, and optimality are neither claimed nor needed. We do not formalize the laser method, and we do not prove the theorem of (Alman et al. 2025) that connects feasibility to ω . That theorem is assumed, as a single named axiom, and §4 describes how that assumption is kept narrow and visible.

2. The object being certified

Combination-loss analysis is parameterized by a maximum recursion level ℓ^* . An instance at level ℓ^* over a base q determines a finite rooted tree whose nodes carry probability distributions, and a feasibility predicate over that assignment. A *certificate* is the assignment, written down exactly.

2.1 Bytes as the object of study

The certificate is a canonical JSON document under RFC 8785 (Rundgren, Jordan, and Erdtman 2020): numbers appear only as exact rationals in a restricted grammar, object keys are sorted, and no formatting freedom remains. Canonicity makes the byte string itself, not a parse of it, the identity of the artifact: a theorem quantified over bytes is then a theorem about exactly the object identified by the published digest.

The artifact discussed throughout is

```
q = 5, l* = 3
5,779 nodes, 762 maximum-entropy blocks
1,485,828 canonical bytes
declared precision 64 bits
sha256 55148017090a8883ab18bbd1316196fadc32b2f5f41cbf751d838d5c334f895f
```

2.2 Why the feasibility test is not a numeric comparison

The predicate to be decided is an inequality between real quantities: sums of entropies, logarithms, and a maximum-entropy term that has no closed form. A floating-point evaluation of such an inequality is not a proof of it, and neither is an exact evaluation that computes the wrong side of a rounding.

The development therefore works with *directed* rational bounds throughout. For each real quantity x the checker computes a rational $\underline{x}$ with $\underline{x} \leq x$, or $\bar{x} \geq x$, according to the direction in which that quantity enters the final inequality. The conclusion

$$\underline{E}_{\text{total}} + \underline{M}_{\text{total}} \Omega \geq 2^{\ell^*-1} \overline{\log_2(q+2)}$$

then implies the corresponding statement about the real quantities, because every substitution moved the inequality in the safe direction.

Direction is carried in the Lean types. `LowerBound` and `UpperBound` are distinct, there is no ambiguous `log2`, and an expression that mixes them incorrectly does not typecheck. The soundness argument for the whole checker is a composition of one-line direction arguments, and encoding direction in the types lets the elaborator check that composition.

3. The development

The Lean 4 (Moura and Ullrich 2021) development is organized into five components, on Mathlib (The mathlib Community 2020) for the real-analysis foundation:

Component	Lines	Role
Spec	2,025	transcribes A.1–A.10 and defines ω
Certificate	1,316	the directed checker and its soundness proofs
Numeric	1,074	enclosures, directed logarithms, entropy
Schema	1,011	a total canonical decoder over <code>ByteArray</code>
Util	127	supporting lemmas

Generated per-certificate modules are excluded from that count; they contain the artifact’s bytes and two theorem statements, and no mathematics.

3.1 The decoder is inside the proof

`MatrixMath.Schema.decodeOmega` is a total, resource-bounded parser from `ByteArray` to a typed certificate, returning an error rather than diverging or panicking on malformed input. It enforces the §6.2 numeric grammar in Lean rather than assuming it: a zero denominator, a non-coprime pair, a negative zero, a leading zero and a `+` sign each have a negative fixture, and Lean rejects all five with the same stable code the reference implementation returns.

This design decision separates the development from a conventional formalization. It would be far easier to transcribe a certificate into Lean literals and prove a theorem about the transcription. The resulting theorem would be true and would not say what a reader wants: it would leave the transcription itself — typically an additional generated artifact — inside the trusted base. Quantifying over bytes removes that step entirely.

3.2 The maximum-entropy term

One quantity in the objective, the maximum entropy $H_D^{\max}(\rho)$ over a transportation polytope, is not computable in closed form. It is handled by the standard certificate device: the artifact carries a witness y together with dual potentials, and the checker verifies rather than solves.

This is the only place in the development where a real supremum must be bounded, so we describe it in detail.

$H_D^{\max}(\rho)$ is the maximum entropy over the transportation polytope of distributions with prescribed marginals. It appears in the objective *negated*, so a sound evaluation needs an **upper** bound on it — which is the hard direction, since exhibiting a feasible point gives a lower bound for free.

The bound comes from Gibbs’ inequality in its dual form, proved in the development from Mathlib’s foundations rather than assumed. Under the checked residual condition, the development proves the exact enclosure

$$H(y) \leq H_D^{\max}(\rho) \leq H(y) + 2\varepsilon.$$

The artifact supplies y and the dual potentials, and the checker verifies three things: that y has the prescribed marginals, that the residual between the potentials and the witness is bounded by a slack ε the artifact also declares,

and that ε is non-negative. No optimization is performed by the checker: it confirms the supplied witness, which is what keeps this step inside the certificate paradigm.

Two producer-side decisions make the verification exact rather than approximate. The marginal constraint is solved over $\mathbb{Q}$ by a marginal-preserving rationalization, so $Ay = b$ is an identity rather than an inequality to be established numerically. And ε is *computed* against an independent third implementation of the directed logarithm rather than inherited from the solver that produced y , so a defect in that solver cannot silently supply its own error bound.

The corresponding Lean lemma is that replacing each $H_D^{\max}$ by a certified upper bound can only lower the conservatively evaluated total, so satisfaction of the directed condition implies satisfaction of the real one — the same direction argument as §2.2, applied to the one term that has no closed form.

4. One project axiom, plus the CN evaluation axiom

4.1 The statement

```
noncomputable def mulRank (n : ℕ) : ℕ :=
  sInf {r | Nonempty (MulDecomposition n r)}

noncomputable def omegaExponent : ℝ :=
  iInf fun n : {m : ℕ // 2 ≤ m} => Real.logb (n : ℕ) (mulRank n)

axiom AX1_combination_loss :
  ∀ {q levels : ℕ} {Ω : ℝ},
    CombinationLossFeasible q levels Ω → omegaExponent ≤ Ω
```

Both sides of the axiom are definitions; the argument of this section rests on that fact.

omegaExponent is defined, not opaque. A `MulDecomposition n r` is r rank-one bilinear terms over $\mathbb{Q}$ summing to the $n \times n$ matrix product — Strassen’s normal form — `mulRank n` is the least such r , and the exponent is the standard infimum characterization $\inf_{n \geq 2} \log_n R(n)$. The decomposition set is proved inhabited by the schoolbook n^3 witness, ranks are proved positive, and the infimum is over a nonempty range bounded below, so the constant is not a junk-value artifact of Mathlib’s total functions. The definition is over $\mathbb{Q}$; the cited theorem holds over every field, so the axiom asserts the weaker specialization. The equivalence of the infimum characterization with the big-O form is classical and not needed by any theorem here. ADR 0013 records these choices.

CombinationLossFeasible is defined, not opaque. It is a Lean predicate that transcribes A.1–A.10 — the rose tree that keeps same-shape nodes distinct by position, the domain constraints, the mass recursion, the split distributions, the retained exponents, the local matrix sizes, and the objective — as a written-down mathematical object, for every ℓ^* rather than for the level the instance happens to use.

The distinction matters. An axiom of the shape `Opaque1 → Opaque2` asserts a relation between things the reader cannot inspect, and can be satisfied vacuously; an axiom relating an opaque constant to a definition still leaves one end uninspectable. An axiom both of whose sides are definitions asserts something checkable against the cited literature at both ends: a reader who disagrees with the formalization of the problem can read `Spec/Instance.lean` and name the clause, and a reader who disagrees with the formalization of the exponent can read `Spec/OmegaExponent.lean` and name the choice. The axiom is where trust enters; the definitions make that trust auditable.

4.2 On assuming the interesting theorem

A natural objection is that this division of labour is backwards: we prove that a certificate satisfies some inequalities, and assume the theorem that makes satisfying them mean anything. The mathematics is assumed and the arithmetic is proved.

We think this is nonetheless the correct allocation, for reasons tied to how these results are produced and read.

The theorem of (Alman et al. 2025) is published and amenable to conventional human review. The certificate is 1,485,828 bytes of exact rationals encoding 5,779 distributions, and the assertion that it satisfies A21 is a claim about a

computation over that object. The two halves of a combination-loss result therefore have different exposure to human scrutiny: the theorem can be read directly, whereas certificate feasibility is naturally checked by computation.

Formal verification adds the most where human review is weakest. Machine-checking the theorem while leaving the computation unchecked would get this backwards. We therefore isolate the refereed mathematics in a single named axiom and machine-check everything downstream of it.

4.3 The soundness chain

The generated theorem for an artifact has two parts. The first is a closed Boolean evaluation over the bytes:

```
theorem omegaCert_55148017... :
  accepts0megaDigest Limits.default bytes_55148017
    omegaClaim_55148017 omegaDigest_55148017 = true := by
  native_decide
```

`accepts0megaDigest` conjoins three conditions — that the checker accepts these bytes, that the Ω they carry is the one named, and that the bytes’ SHA-256 **is the published digest** — into a single Boolean, so that one native evaluation suffices and the theorem is checkably about exactly the artifact the digest identifies. The byte literal’s binding to the published file is decided inside the evaluation, not by a generator step outside the development. The second part discharges the mathematics:

```
theorem omegaResult_55148017... :
  omegaExponent ≤ ((omegaClaim_55148017 : Rat) : Real) :=
  omega_le_of_accepts0megaDigest omegaCert_55148017...
```

`omega_le_of_accepts0mega` is proved, and composes three proved steps with one axiom; Figure 1 shows the chain.

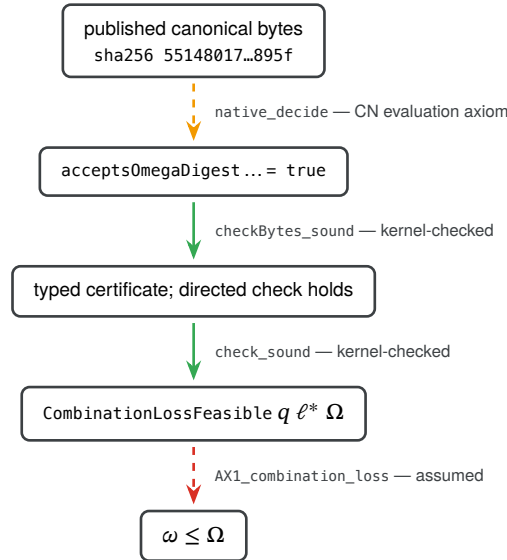


Figure 1: The soundness chain from published bytes to a bound on ω . Solid arrows are kernel-checked theorems; dashed arrows are the two declared axioms (the CN native evaluation and AX1).

`check_sound` is the mathematical core: it turns “the rational directed condition holds” into “this data is feasible for the cited real problem”. An earlier revision of this development discharged that step with a hypothesis named `FeasibilityBridge`, which amounted to assuming the interesting half. It is now a theorem, and `check_sound` itself depends only on Lean’s standard axioms.

4.4 The policy, and what it excludes

`#print axioms` enumerates what a term depends on. The development checks that output mechanically instead of reading it by eye. For the certificate discussed here:

```

omegaCert_55148017...  propext, Classical.choice, Quot.sound,
                        omegaCert_55148017..._native.native_decide.ax_1_1
omegaResult_55148017... propext, Classical.choice, Quot.sound,
                        AX1_combination_loss,
                        omegaCert_55148017..._native.native_decide.ax_1_1

```

The policy admits `propext`, `Classical.choice` and `Quot.sound` — Lean’s standard mathematical axioms, on which Mathlib rests — plus `AX1_combination_loss`, plus *at most one* certificate-specific native-evaluation axiom whose type asserts the closed checker result for that artifact. Anything else is a release failure: a second project axiom, `sorryAx`, or an undeclared appeal to the compiler.

The one-axiom bound on native evaluation is the reason `acceptsOmegaDigest` is a single Boolean rather than a conjunction of separate checks. A development that evaluated four Booleans would import four native axioms, and the policy would have to reason about their conjunction; folding the check into one keeps the admissible set finite and syntactically checkable.

5. Two profiles and two trusted bases

Not every certificate can be decided by the kernel. The development therefore declares two profiles, and a result carries the one it achieved:

CK, kernel-certified. The kernel reduces the Boolean itself. The trusted base is Lean’s kernel and the axioms above. This is achieved for the tensor decompositions in the same platform — a 47-term decomposition of T_4 over $\mathbb{F}_2$ is CK, with `#print axioms` listing only `propext` and `Quot.sound`.

CN, native-certified. The Boolean is evaluated by compiled code and the result is imported as one auditable axiom. The trusted base additionally includes the Lean compiler, its runtime, and its bignum implementation.

The bound reported here is CN, and structurally so. Every directed bound in the checker is a Lean core `Rat`, whose arithmetic the kernel cannot reduce at this scale; `decide` does not terminate usefully on an object with 5,779 nodes and rationals of hundreds of thousands of digits. This is a property of the representation, not of the mathematics, and the platform refuses a CK request for such a certificate up front with that reason; it does not emit a module that would fail.

5.1 What `native_decide` is

The mechanism behind CN is `native_decide`. It compiles a decidable proposition, runs it, and if the result is `true` introduces an axiom asserting so. Its poor reputation in the Lean community is deserved: soundness bugs have been found in it, and it admits the entire compiler and runtime into the trusted base.

CN is a weaker guarantee than CK. A reader who does not trust the Lean compiler should discount this result accordingly.

We claim only that the discount is bounded, in a way that an unverified checker’s output is not. Three things distinguish the two:

The mathematics is kernel-checked. The decoder, the directed arithmetic, the soundness of the check, and the reduction to the cited problem are ordinary Lean proofs reduced by the kernel. `native_decide` is not asked to establish any of it.

The native component is one closed Boolean with a written-down type. It is not a program whose behaviour must be characterized; it is the single proposition `acceptsOmega Limits.default bytes claim = true`, which a reader can inspect and which the policy of §4.4 requires to be unique. A compiler bug that invalidates this result must produce `true` for that specific evaluation on those specific bytes.

It is independently falsifiable. The same Boolean is decided by a Rust implementation written separately against the same specification, and the two must agree before a theorem is emitted. That is not a proof — two implementations can share a misreading of the specification, and §6 records that this is the residual risk — but a compiler defect would have to be mirrored by an unrelated defect in an unrelated language to survive it.

In summary, CN converts *trust this checker* into *trust the Lean compiler on one enumerated evaluation, given that the surrounding mathematics is kernel-checked and the same evaluation is corroborated independently*. The residual trust is smaller, but it is not zero. Reaching CK for a certificate of this size would require kernel-reducible arithmetic at a scale Lean’s core Rat does not currently support. We do not claim to have solved for that.

6. What the theorem does not say

Four limitations bound the claim.

AX1 is assumed. The theorem of (Alman et al. 2025) connecting feasibility to ω is not proved here. Formalizing the laser method is an explicit non-goal. A reader who doubts that theorem should apply the same doubt to this bound; isolating the axiom makes that dependence explicit.

The bound is an upper bound. Nothing here constrains the true value of ω from below, and the development says nothing about whether the bound is tight.

The formalizations could be wrong. `CombinationLossFeasible` is a human transcription of a published problem statement, and `omegaExponent` is a human transcription of a standard definition. If either transcribes a *different* object — weaker, or merely adjacent — then the theorem is true but about the wrong object. Proving cannot eliminate this risk, only make it auditable, which is why both are defined rather than opaque, why the development states the problem for every ℓ^* instead of specializing to the instance in hand, and why the definition of the exponent carries its inhabitation and positivity lemmas. Generalizing the predicate across ℓ^* found a real indexing defect that the level-two case could not exhibit, which is the kind of evidence available for this class of risk. For the exponent, the specific choices — field $\mathbb{Q}$, the infimum characterization — are recorded in ADR 0013 with their justifications.

Under CN the compiler is trusted. See §5.

7. Result

For the artifact with digest 55148017090a8883ab18bbd1316196fadc32b2f5f41cbf751d838d5c334f895f:

$$\omega \leq \frac{10935605172023554189}{2^{62}} = 2.371281376 \dots$$

accepted from the canonical bytes by the Lean development under profile CN, with `AX1_combination_loss` as the only project axiom and one certificate-specific native-evaluation axiom, both enumerated by `#print axioms` and both accepted by the policy of §4.4.

Figure 2 places the result in the reported sequence of bounds.

Two comparisons follow, and they should be kept separate.

Against (Alman et al. 2025) the comparison is at the same recursion level and uses the same published A.1–A.21 feasibility formulation; relative to the displayed threshold, ours is lower by 5.76×10^{-5} .

Against (Dupont et al. 2026) it is not. That result is obtained at level four, where the optimization carries approximately 7 million parameters against the 25 thousand reported for level three, and it remains lower than ours by 1.04×10^{-4} . We make no claim on the state of the art.

Every bound in the sequence rests on a computer-assisted argument. The highlighted point additionally carries a Lean theorem over its certificate bytes.

7.2 On the search, and what this artifact does not claim

A reader weighing the level-three comparison should know what produced the point, to the extent it bears on the claim. The search ran over the **unrestricted** per-node space rather than a symmetric subspace that ties nodes sharing a level and shape — 12,887 simplex and box degrees of freedom across the 5,779-node tree, against 417 for the tied version. The unrestricted space is what A.2 always permitted; the symmetric restriction was an implementation choice in prior work, and removing it required no change to the published formulation.

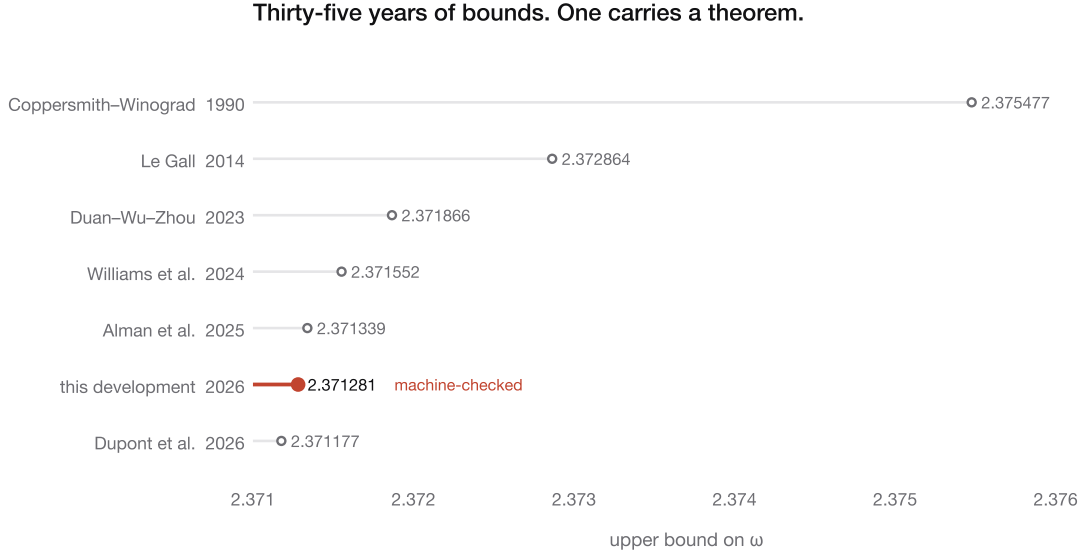


Figure 2: Published bounds on ω since 1990. To our knowledge, the highlighted point is the only entry carrying a proof-assistant theorem over its certificate bytes.

While contemporary results of this kind typically rest on large-scale distributed compute, the search that produced this point was lightweight: the certified endpoint’s full lineage is roughly 1.1 million optimizer steps — about forty core-hours in total — computed on a single consumer system-on-chip (Apple M1 Max, 8 performance cores, 64 GB) across four chained generations, the longest of which ran about seventeen hours of wall clock.

Beyond the hardware envelope above, the search methodology is not part of this artifact, and the asymmetry is deliberate and honest in both directions: no claim in this paper depends on how the point was found, and, symmetrically, this paper makes **no further claims about the search** — not that it is novel, converged, or superior to any other. The certificate is the entire argument. A reader who distrusts an undisclosed search is invited to distrust it completely; the theorem works as it does.

8. Related work

Formal verification of computer-assisted mathematics has an established pattern: a result whose proof includes a large computation is reduced to a kernel-checked statement, sometimes years after the original (Gonthier 2008; Hales et al. 2017). The reduction is usually the harder half of the work, and the usual obstacle is that the original computation was not designed to be checked.

The matrix multiplication line has not yet had that treatment. (Dupont et al. 2026) reports that verification code will be released, which would be a substantial step and is a different artifact from a proof-assistant theorem: it moves the question from *trust this program* to *trust this other program*.

The device used here — a producer that may be arbitrarily sophisticated and is never trusted, feeding a consumer that must be independently intelligible — is proof-carrying code (Necula 1997) applied to a mathematical bound rather than to a program. The specific contribution is the placement of the trust boundary: at the bytes, so that the decoder is inside the proof, and at one axiom whose hypothesis and conclusion are both definitions.

9. Conclusion

The upper bound on ω has been improved eight times in thirty-five years, and every improvement rests on a computation no proof assistant has examined. This is understandable: the computations are large, the mathematics around them is hard, and the incentive has been to improve the bound, not to re-establish it.

The bound reported here is below a published level-three result and above the current level-four record. The numerical ordering may change; the verification claim is tied to one immutable artifact and does not.

The verification result is the durable claim. We have shown that the gap between computer-assisted and machine-checked can be closed at reasonable cost, and the reason transfers to other results: the artifact was designed to be checked. A canonical byte format, exact rationals in a restricted grammar, witnesses supplied for every quantity without a closed form, and direction carried in the type system are all producer-side decisions, and each one removes work from the verifier. The usual obstacle to formalizing a computer-assisted result is that the original computation was not built with this in mind; that obstacle is avoidable, not intrinsic.

What we have not shown is that the same verification path is practical at the level-four frontier. That question involves a much larger certificate and is outside the scope of this level-three result. No level-four feasibility or throughput claim is needed for the theorem reported here.

Appendix

A. Reproducing the verification

The two public objects are:

- **Repository:** <https://github.com/bsd-developer/matrix-math-publish>
- **Artifact DOI:** [10.5281/zenodo.22101463](https://doi.org/10.5281/zenodo.22101463)

The claim of §7 is checkable without trusting this paper or its author once the canonical artifact has been retrieved. The artifact is a byte string with the digest stated in §7, and the theorem quantifies over that byte string.

The certificate ships in the repository under `artifacts/` and as the Zenodo deposit named above; both carry the digest stated in §7. The release practice was that a successful verification from a fresh clone on a clean machine preceded the publication of either link.

```
git clone https://github.com/bsd-developer/matrix-math-publish.git
cd matrix-math-publish
```

```
# Toolchains are pinned; this reports any mismatch rather than proceeding.
```

```
just doctor
```

```
just build
```

```
# After retrieving the release artifact to its recorded path:
```

```
CERT=artifacts/omega-55148017.certificate.json
```

```
# The artifact is identified by its bytes, so check them first.
```

```
shasum -a 256 "$CERT"
```

```
# 55148017090a8883ab18bbd1316196fadc32b2f5f41cbf751d838d5c334f895f
```

```
# Exact rational check, then the Lean theorem over the same bytes.
```

```
just verify "$CERT"
```

`just verify` decodes the certificate, decides the directed A21 condition in exact rational arithmetic, rebuilds the generated module, and reports the profile together with the `#print axioms` output. Runtime is machine-dependent and is not part of the mathematical claim.

A reader who wants only the arithmetic, without building Lean, may pass `--skip-lean`; the result is then reported as **XC**, which §5 explains is a development cross-check and is never reportable as certified.

B. Axiom-policy summary

The result-local assurance record produced by the verification command contains the following dependency classes; the archived record must retain the full, unabridged declaration names and output:

profile	CN
module	MatrixMath.Generated.Omega_55148017090a8883
declaration	omegaResult_55148017090a8883...f895f

```
project_axioms [MatrixMath.AX1_combination_loss]

'omegaCert_55148017...895f' depends on axioms:
  [propext, Classical.choice, Quot.sound,
   omegaCert_55148017...895f._native.native_decide.ax_1_1]

'omegaResult_55148017...895f' depends on axioms:
  [propext, Classical.choice, AX1_combination_loss, Quot.sound,
   omegaCert_55148017...895f._native.native_decide.ax_1_1]
```

Three standard axioms, one project axiom, and one certificate-specific native-evaluation axiom. §4.4 states the policy this satisfies; anything further is a release failure.

C. Where each clause of the problem is transcribed

§6 records that the residual risk is transcription: if `CombinationLossFeasible` states a different problem from the cited one, the theorem holds of the wrong object, and auditability is the only mitigation. The project specification organizes the transcription as Appendix §§A.1–A.10; within those sections the cited equations are numbered (A1)–(A21). This table gives a Lean anchor for every numbered equation.

Equation	Content	Lean anchor
A1	maximum-entropy penalty	penalty
A2	root-child masses	childMasses
A3	interior-child masses	childMasses
A4	zero-shape dual distributions	SupportVec.dual, beta0f
A5	concatenated split mixtures	betaRegion
A6	region mixture	beta0f_posBranch
A7	level-two split distributions	beta0f
A8	root η_Y	etaY, etaYMix
A9	root η_Z	etaZ, etaZMix
A10	root regional minimum	eRootRegion
A11	root retained exponent	eRoot
A12	interior η_Y	etaY, etaYMix
A13	interior η_Z	etaZ, etaZMix
A14	interior regional exponents	eInteriorRegion
A15	level retained exponent	eLevel
A16	level-two coordinate exponents	levelTwoExponent
A17	level-two minimum	eTwo, eTwoSum
A18	zero-shape local matrix sizes	localSize
A19	level-two local matrix sizes	localSize
A20	total retained exponent and matrix size	eTotal, mTotal
A21	final feasibility inequality	Feasible

D. Certificate shape

```
q = 5, l* = 3
5,779 nodes, 762 maximum-entropy blocks
1,485,828 canonical bytes, RFC 8785
declared precision 64 bits, rational width 48 bits
omega = 10935605172023554189 / 2^62
```

E. On the separation, and contact

The separation of the search from the certificate is deliberate, as an exercise in reporting a computational result so that nothing about its discovery has to be trusted. The search apparatus — its design, its measurements, and its source

— is therefore not published with this artifact. The author is open to sharing details with serious researchers and potential collaborators: `bsd.developer@proton.me`.

F. Note on AI assistance

AI systems were used substantially in this work — as engineering, analysis, and writing assistants — under the author’s direction; experimental priorities and design decisions were the author’s. The paper is designed so that this requires no trust: no claim depends on who, or what, wrote the search, the certificate producer, or any unpublished code. The theorem is checkable from the released repository and artifact alone.

Bibliography

- Alman, Josh, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2025. “More Asymmetry Yields Faster Matrix Multiplication.” In *ACM-SIAM Symposium on Discrete Algorithms (SODA)*.
- Coppersmith, Don, and Shmuel Winograd. 1990. “Matrix Multiplication via Arithmetic Progressions.” *Journal of Symbolic Computation* 9 (3): 251–80.
- Duan, Ran, Hongxun Wu, and Renfei Zhou. 2023. “Faster Matrix Multiplication via Asymmetric Hashing.” In *64th IEEE Symposium on Foundations of Computer Science (FOCS)*.
- Dupont, Emilien, Marvin Eisenberger, Borislav Kozlovskii, Abbas Mehrabian, Francisco J. R. Ruiz, Abigail See, Renfei Zhou, Josh Alman, Virginia Vassilevska Williams, and Matej Balog. 2026. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” <https://arxiv.org/abs/2608.16884>.
- Gall, François Le. 2014. “Powers of Tensors and Fast Matrix Multiplication.” *Proceedings of ISSAC*.
- Gonthier, Georges. 2008. “Formal Proof—the Four-Color Theorem.” *Notices of the AMS* 55 (11): 1382–93.
- Hales, Thomas, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Le Truong Hoang, Cezary Kaliszyk, et al. 2017. “A Formal Proof of the Kepler Conjecture.” In *Forum of Mathematics, Pi*. Vol. 5.
- Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction – CADE 28*, 625–35. https://doi.org/10.1007/978-3-030-79876-5_37.
- Necula, George C. 1997. “Proof-Carrying Code.” In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19. <https://doi.org/10.1145/263699.263712>.
- Rundgren, Anders, Bret Jordan, and Samuel Erdtman. 2020. “JSON Canonicalization Scheme (JCS).” *RFC 8785, IETF*.
- Strassen, Volker. 1969. “Gaussian Elimination Is Not Optimal.” *Numerische Mathematik* 13: 354–56.
- The mathlib Community. 2020. “The Lean Mathematical Library.” In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 367–81. <https://doi.org/10.1145/3372885.3373824>.
- Williams, Virginia Vassilevska, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2024. “New Bounds for Matrix Multiplication: From Alpha to Omega.” In *ACM-SIAM Symposium on Discrete Algorithms (SODA)*.